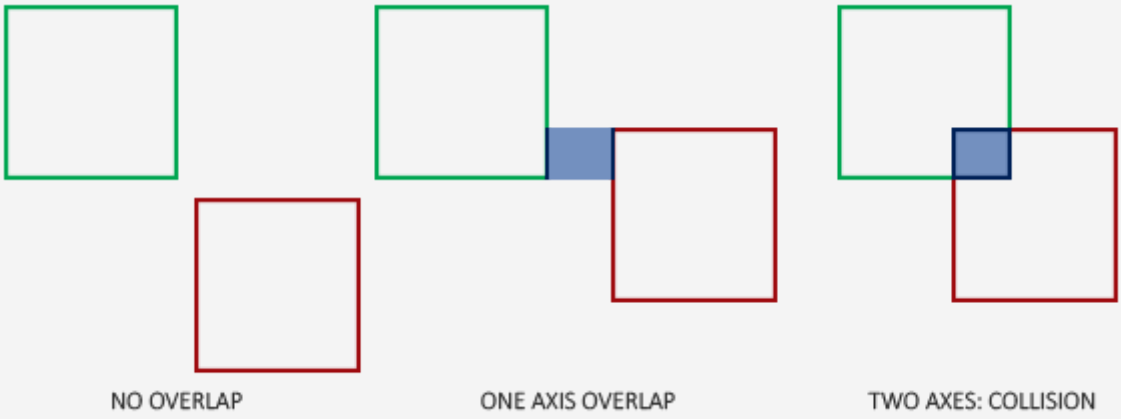
Gestion des collisions

En jeux vidéo, la gestion des collisions est essentielle pour déterminer les interactions entre les objets. Par exemple, lorsqu'un personnage entre en contact avec un mur ou un ennemi, une collision est détectée et une action appropriée est déclenchée (comme arrêter le mouvement ou perdre de la vie). La logique de détection des collisions peut varier en complexité, allant de simples vérifications de chevauchement de rectangles à des calculs plus avancés impliquant des formes irrégulières.

Dans le cas de notre projet, la collision sera gérée en vérifiant si les coordonnées de l'objet en mouvement (par exemple, une voiture) dépassent les limites définies par les autres objets (comme des murs ou des obstacles). Si une collision est détectée, nous pourrons réagir en ajustant la position de l'objet ou en déclenchant un événement spécifique.

Un peu comme avec la collision avec le curseur de la souris, nous allons vérifier si les coordonnées de notre objet en mouvement se trouvent à l'intérieur des limites d'un autre objet. Si c'est le cas, une collision est détectée.



Pour ce faire, deux rectangles se chevauchent si :

1. Le bord droit du premier rectangle est à droite du bord gauche du second rectangle.
2. Le bord gauche du premier rectangle est à gauche du bord droit du second rectangle.
3. Le bord inférieur du premier rectangle est en dessous du bord supérieur du second rectangle.
4. Le bord supérieur du premier rectangle est au-dessus du bord inférieur du second rectangle.

```
SI le bord droit du rectangle A (positionX + largeur) > le bord gauche du
rectangle B (positionX)
ET le bord gauche du rectangle A (positionX) < le bord droit du rectangle B
(positionX + largeur)
ET le bord inférieur du rectangle A (positionY + hauteur) > le bord supérieur du
rectangle B (positionY)
ET le bord supérieur du rectangle A (positionY) < le bord inférieur du rectangle B
(positionY + hauteur) ALORS
    IL Y A COLLISION
SINON
    IL N'Y A PAS COLLISION
FIN SI
```

```
/**
 * Vérifie si deux rectangles se chevauchent (collision).
 * @param {Object} rectangleA - Le premier rectangle avec les propriétés
positionX, positionY, largeur, hauteur.
 * @param {Object} rectangleB - Le second rectangle avec les mêmes propriétés.
 * @return {boolean} - Retourne true s'il y a collision, sinon false.
 */
function verifierCollision(rectangleA, rectangleB) {
    if (
        rectangleA.positionX < rectangleB.positionX + rectangleB.largeur &&
        rectangleB.positionX < rectangleA.positionX + rectangleA.largeur &&
        rectangleA.positionY < rectangleB.positionY + rectangleB.hauteur &&
        rectangleB.positionY < rectangleA.positionY + rectangleA.hauteur
    ) {
        return true; // Il y a collision
    }
    return false; // Il n'y a pas collision
}
```

Exemple d'utilisation

```
let voiture = {
    positionX: 50,
    positionY: 50,
    largeur: 30,
    hauteur: 15,
};
let mur = {
    positionX: 40,
    positionY: 40,
    largeur: 10,
    hauteur: 100,
};

let arrivee = {
    positionX: 200,
    positionY: 200,
    largeur: 50,
    hauteur: 50,
};

let touche = null;
// VERSION ENCORE PLUS SIMPLE - Pour des débutants
function toucheEnfoncee(evenement) {
    touche = evenement.key;
}

function deplacerVoiture(deplacementX, deplacementY) {
    // Sauvegarder position actuelle
    let ancienneX = voiture.positionX;
    let ancienneY = voiture.positionY;
```

```
    // Essayer le nouveau mouvement
    voiture.positionX += deplacementX;
    voiture.positionY += deplacementY;

    // Si collision, revenir en arrière
    if (verifierCollision(voiture, mur)) {
        voiture.positionX = ancienneX;
        voiture.positionY = ancienneY;
    }
}

function dessinerTout() {
    oContexte.clearRect(0, 0, nLargeur, nHauteur);
    oContexte.fillStyle = "blue";
    oContexte.fillRect(voiture.positionX, voiture.positionY, voiture.largeur,
voiture.hauteur);
    oContexte.fillStyle = "brown";
    oContexte.fillRect(mur.positionX, mur.positionY, mur.largeur, mur.hauteur);
    oContexte.fillStyle = "green";
    oContexte.fillRect(arrivee.positionX, arrivee.positionY, arrivee.largeur,
arrivee.hauteur);
}

function boucleJeu() {
    dessinerTout();

    if (touche === "ArrowRight") {
        deplacerVoiture(5, 0);
    } else if (touche === "ArrowLeft") {
        deplacerVoiture(-5, 0);
    } else if (touche === "ArrowUp") {
        deplacerVoiture(0, -5);
    } else if (touche === "ArrowDown") {
        deplacerVoiture(0, 5);
    }

    if (verifierCollision(voiture, arrivee)) {
        console.log("Arrivée atteinte !");
        sEtatJeu = "fin";
    }
}
```

Types de réactions aux collisions

Selon le contexte du jeu, différentes réactions sont possibles lors d'une collision :

```
// 1. COLLECTE - L'objet disparaît et augmente le score
let pieces = [
    { positionX: 100, positionY: 100, largeur: 20, hauteur: 20, valeur: 10 },
    { positionX: 150, positionY: 80, largeur: 20, hauteur: 20, valeur: 15 },
];
```

```
function verifierCollectionPieces() {
    for (let i = pieces.length - 1; i >= 0; i--) {
        if (verifierCollision(joueur, pieces[i])) {
            score += pieces[i].valeur;
            pieces.splice(i, 1); // Retirer la pièce du tableau
        }
    }
}

// 2. DOMMAGES - Le joueur perd de la vie
// Nous verrons les tableaux la semaine prochaine
let ennemis = [
    { positionX: 200, positionY: 150, largeur: 25, hauteur: 25, degats: 10 },
    { positionX: 300, positionY: 200, largeur: 25, hauteur: 25, degats: 15 },
];

function verifierCollisionEnnemis() {
    for (let i = 0; i < ennemis.length; i++) {
        let ennemiAVerifier = ennemis[i];
        if (verifierCollision(joueur, ennemiAVerifier)) {
            joueur.vie -= ennemiAVerifier.degats;

            // Repousser le joueur pour éviter les dégâts répétés
            joueur.positionX -= 10;
        }
    }
}

// 3. TÉLÉPORTATION - Changement de position
let portail = { positionX: 300, positionY: 200, largeur: 30, hauteur: 30 };

function verifierTeleportation() {
    if (verifierCollision(joueur, portail)) {
        joueur.positionX = 50; // Retour au début
        joueur.positionY = 50;
        console.log("Téléportation activée !");
    }
}
```

Optimisation pour les débutants

Voici une fonction simplifiée qui combine tout :

```
/**
 * Fonction principale de gestion des collisions
 * Appelée dans la boucle de jeu principale
 */
function gererToutesLesCollisions() {
    // 1. Vérifier les obstacles (bloque le mouvement)
    if (verifierCollisionObstacles(joueur)) {
        // Le mouvement est déjà annulé dans deplacerJoueur()
```

```
    }

    // 2. Vérifier la collecte d'objets
    verifierCollectionPieces();

    // 3. Vérifier les ennemis (dommages)
    verifierCollisionEnnemis();

    // 4. Vérifier la ligne d'arrivée
    if (verifierCollision(joueur, arrivee)) {
        console.log("Niveau terminé !");
        niveauSuivant();
    }

    // 5. Vérifier les téléporteurs
    verifierTeleportation();
}

// Dans la boucle de jeu principale
function boucleJeu() {
    // Effacer l'écran
    oContexte.clearRect(0, 0, nLargeur, nHauteur);

    // Dessiner tous les objets
    dessinerTout();

    // Gérer toutes les collisions
    gererToutesLesCollisions();
}
```